A menudo se desea encontrar el valor aproximado de una cantidad, en vez del valor exacto con la intención de comparar un número con otro. Emplearemos cálculo elemental para obtener los resultados que necesitamos. Una de las ventajas del uso de la notación $O$ que introduciremos en esta sección es que podemos estimar el crecimiento de una función sin preocuparnos por las constantes de multiplicación o los términos de menor orden.

**Definición:**

Sean $f$, $g : \mathbb{N} \longrightarrow \mathbb{R}$. Decimos que $f(n)$ es $O(g(n))$, si existen constantes $C \in \mathbb{R}_{>0}$ y $k \in \mathbb{N}$ tales que $|f(n)| \leq C|g(n)|$, $\forall n \in \mathbb{N}$, donde $n \geq k$.
Notación: $f \in O(g)$.

## Ejemplo 1:

Si $f$, $g : \mathbb{N} \longrightarrow \mathbb{R}$, dadas por $f(n) = 5n^2 + 3n + 1$, $g(n) = n^2$. Entonces:

$$|f(n)| = |5n^2 + 3n + 1| = 5n^2 + 3n + 1 \leq 5n^2 + 3n^2 + n^2 = 9n^2 = 9|g(n)|.$$

Tenemos que, para todo $n \geq 1 (= k)$, $|f(n)| \leq C|g(n)|$, con $C = 9$ y $f \in O(g)$.

## Ejemplo 2:

Si $f$, $g : \mathbb{N} \longrightarrow \mathbb{R}$, dadas por $f(n) = 3n^3 + 7n^2 - 4n + 2$, $g(n) = n^3$. Aqui:
$|f(n)| = |3n^3 + 7n^2 - 4n + 2| \leq |3n^3| + |7n^2| + |-4n| + |2| \leq$
$3n^3 + 7n^3 + 4n^3 + 2n^3 = 16n^3 = 16|g(n)|, \forall n \geq 1$ Así, con $C = 16$ y $k = 1$, tenemos que $f \in O(g)$ o bien $f \in O(n^3)$.
Como $7n - 4 > 0$, para $n \geq 1$, podemos escribir:
$n^3 \leq 3n^3 \leq 3n^3 + (7n - 4)n + 2, \forall n \geq 1$ Entonces,
$|g(n)| \leq |f(n)|, \forall n \in \mathbb{N}$, $g \in O(f)$. Así, $O(n^3) = O(g)$, en este caso.

## Obs:

- Si tenemos dos funciones $f(n)$ y $g(n)$, tal que $f(n) \in O(h(n))$ y $g(n) \in O(h(n))$ entonces $f(n)$y $g(n)$ son del mismo orden.
- Cuando $f(n) \in O(g(n))$ y $h(n)$ es una función que tiene valores absolutos más grandes que $g(n)$ para valores grandes de n, se sigue que $f(n) \in O(h(n))$.
- Cuando la notación $O$ es usada, la función $g$ se elige **tan pequeña** como sea posible.

A continuación se listan de menor a mayor complejidad las funciones más utilizadas:

$$1; \log(n); n; nlog(n); n^2; n^3; n^m; m^n (m \geq 2); n!$$

## Ejemplo 3:

Hallar $O(n!)$ y $O(\log(n!))$

*Solución:* La función factorial $f(n) = n!$ se define como $n! = 123...n$ para cada entero positivo $n$ y $0! = 1$.

Se puede obtener una estimación de $n!$ en notación $O$ teniendo en cuenta que cada término el producto es menor que $n$. Por tanto,

$$n! = 123...n \leq nnn...n = n^n.$$

Esta desigualdad muestra que $n! \in O(n^n)$, tomando $C = 1$ y $k = 1$.

Aplicando logaritmos en la desigualdad anterior, se obtiene:

$$\log(n!) \leq n \log(n).$$

Esto implica que $\log(n!) \in O(n \log n)$, de nuevo considerando $C = 1$ y $k = 1$.

A menudo es necesario estimar el crecimiento de la suma y el producto de dos funciones.
¿Qué puede decirse si se conocen estimaciones en notación $O$ para cada una de las dos funciones? Para ver qué clase de estimación se obtiene para la suma y el producto de dos funciones, supongamos que $f_1 \in O(g_1)$ y $f_2 \in O(g_2)$).
De la definición, se sigue que hay cuatro constantes $C_1$, $C_2$ , $k_1$ y $k_2$ tales que

$$|f_1(n)| \leq C_1|g_1(n)|$$

cuando $n \geq k_1$ y

$$|f_2(n)| \leq C_2|g_2(n)|$$

cuando $n \geq k_2$.
Para estimar la suma de $f_1$ y $f_2$, observar que

$$|(f_1 + f_2)(n)| = |f_1(n) + f_2(n)| \leq |f_1(n)| + |f_2(n)|$$

utilizando la desigualdad triangular.

Cuando $n$ es mayor que $k_1$ y $k_2$, se deduce de las desigualdades para $|f_1(n)|$ y $|f_2(n)|$ que

$$|f_1(n)| + |f_2(n)| \leq C_1|g_1(n)| + C_2|g_2(n)|$$
$$\leq C_1|g(n) + C_2|g(n)| = C|g(n)|$$

donde $C = C_1 + C_2$ y $g(n) = \text{máx}(|g_1(n)|, |g_2(n)|)$, siempre que $n \geq k = \text{máx}(k_1, k_2)$.

**Proposición**

Si $f_1 \in O(g_1)$ y $f_2 \in O(g_2)$ entonces

1. $(f_1 + f_2)(n) \in O(\text{máx}\,|g_1(n)|, |g_2(n)|)$
2. $(f_1.f_2)(n) \in O(g_1.g_2)$

## Ejemplo 4:

Dar una estimación en notación $O$ para
$f(n) = 3n\log(n!) + (n^2+3)\log(n)$, donde $n \in \mathbb{N}$.
*Solución:* Primero estimaremos el producto $3n\log(n!)$. Del Ejemplo 3 se sabe que $\log(n!) \in O(n\log n)$. Usando esto y el hecho de que $n \in O(n)$,la proposición anterior afirma que la estimación para
$3n\log(n!) \in O(n^2\log(n))$. Ahora estimamos el producto $(n^2+3)\log(n)$. Como $n^2+3 < 2n^2$ cuando $n \geq 2$. entonces $n^2+3 \in O(n^2)$. Por tanto, $(n^2+3)\log(n) \in O(n^2\log(n))$.
Combinando las dos estimaciones, se ve que $f \in O(n^2\log(n))$.

# Las notaciones OMEGA y TITA (Optativo)

La notación $O$ se utiliza frecuentemente para describir el crecimiento de funciones. pero tiene algunas limitaciones. En particular, cuando $f(x) \in O(g)$, tenemos una *cota superior* en términos de $g$ del tamaño de $f$ para valores grandes de $n$. Sin embargo, la notación $O$ no nos proporciona una *cota inferior* del tamaño de $f$ para valores grandes de $n$. Para esto se utiliza la notación Omega. Cuando queremos dar tanto una cota inferior como una cota superior del tamaño de la función $f$ en términos de una función de referencia $g$ usamos la notación Tita.

**Definición:**

Sean $f$, $g : \mathbb{N} \longrightarrow \mathbb{R}$. Decimos que $f(n)$ **es** $\Omega(g(n))$, si existen constantes $C \in \mathbb{R}_{>0}$ y $k \in \mathbb{N}$ tales que $|f(n)| \geq C|g(n)|$, $\forall n \geq k$.
Notación: $f \in \Omega(g)$.

## Ejemplo:

La función $f(n) = 8n^3 + 5n^2 + 7$ es $\Omega(g)$, donde $g(n) = n^3$. Porque: $f(n) = 8n^3 + 5n^2 + 7 \geq 8n^3$, $\forall n \geq 1$. Esto es equivalente a decir que $g(n) = n^3 \in O(f)$.

## Definición:

Sean $f$, $g : \mathbb{N} \longrightarrow \mathbb{R}$. Decimos que $f(n)$ **es** $\Theta(g(n))$, si $f \in O(g)$ y $f \in \Omega(g)$.
Notación: $f \in \Theta(g)$.

## **Ejemplo**: Encontrar $\Theta$ para la suma de los $n$ primeros números naturales.

*Solución:* Como cada uno de los números en la suma de los primeros $n \in \mathbb{N}$ es menor o igual que $n$, tenemos que

$$1 + 2 + ... + n \leq n + n + ... + n = n^2$$

Se sigue que $1 + 2 + 3 + ... + n \in O(n^2)$, tomando $C = 1$ y $k = 1$.

Para mostrar que $f(n) = 1 + 2 + ... + n$ es Omega de $n^2$ necesitamos encontrar una constante positiva $C$ tal que $f(n) \geq Cn^2$ para $n$ suficientemente grande. Para obtener una cota inferior de esta suma, podemos acotar la segunda mitad de los términos. Sumando sólo los términos mayores que $\lceil n/2 \rceil$, tenemos que

$$1 + 2 + ... + n \geq \lceil n/2 \rceil + (\lceil n/2 \rceil + 1) + ... + n$$

$$\geq \lceil n/2 \rceil + \lceil n/2 \rceil + ... + \lceil n/2 \rceil = (n - \lceil n/2 \rceil + 1)\lceil n/2 \rceil$$

$$\geq (n/2)(n/2) = n^2/4$$

Esto muestra que $f \in \Omega(n^2)$ o usando la notación Tita, que $f \in \Theta(n^2)$.

**Ejercicio**: Encontrar $\Theta$ para $f(x) = a_n x^n + a_{n-1} x^{n-1} + ... + a_0$ donde $a_i \in \mathbb{N}$, $x \in \mathbb{N}$, y $a_n \neq 0$.

# Análisis de algoritmos

## Definición:

**Algoritmo:** Es un conjunto de instrucciones perfectamente expresadas de tal modo que pueden ser ejecutadas por una máquina o una persona, sin apelar a conocimientos adicionales a los que requieran las mismas instrucciones ni al sentido común. La noción de algoritmo es la de un procedimiento mecánico.

**Propiedades:**

1. **Finitud:** debe ser descripto por un texto finito.
2. **Efectividad:** cada paso del algoritmo tiene que poder ser efectuado en la práctica.
3. **Terminación:** tiene que parar en un número finito de pasos.
4. **Determinismo:** la secuencia de pasos tiene que estar unívocamente determinada para cada entrada.

### Ejemplo 5:

Programas de computadora, fórmulas matemáticas, modelo de máquinas abstractas.

Vamos a medir indirectamente la complejidad de un cálculo, midiendo los recursos necesarios para ejecutarlo. Esto se basa en que un cálculo difícil requerirá más recursos que uno de menor dificultad.

### Definición:

La **complejidad** de un cálculo es la cantidad de recursos necesarios para efectuarlo.

### Definición:

La **complejidad temporal** es la cantidad de tiempo necesaria para efectuar el cálculo.

### Definición:

La **complejidad espacial** es la cantidad de espacio de almacenamiento que requiere un cálculo.

La **complejidad de un algoritmo** es función de la longitud de la entrada, con lo cual se puede analizar el peor caso, el mejor caso y el caso promedio. Para una definición formal se adopta un punto de vista pesimista y se define la complejidad temporal del algoritmo como su rendimiento en el **peor caso**. Por comportamiento de un algoritmo en el peor caso entendemos el mayor número de operaciones que hace falta para resolver el problema dado utilizando el algoritmo para unos datos de entrada de un determinado tamaño. Los análisis del peor caso nos dicen cuántas operaciones tienen que realizar los algoritmos para garantizar que producirán una solución. La complejidad de un problema es la complejidad del algoritmo más sencillo que lo resuelve.

La complejidad en tiempo de un algoritmo se puede expresar en términos del número de operaciones que realiza el algoritmo cuando los datos de entrada tienen un tamaño particular. Las operaciones utilizadas para medir la complejidad pueden ser la **comparación de enteros**, la **suma**, **multiplicación** o **división** de enteros, así como cualquier otra operación básica.

En este curso, para obtener la función de complejidad **sólo contaremos las comparaciones** hechas por el algoritmo.

## Ejemplo 6:

Entrada: x:=3
FOR I:=1 to N
x:=3.x
END

Sólo tenemos las comparaciones de FOR. Tenemos $f(N) = N + 1$ comparaciones:
$I = 1 \leq N$;
$I = 2 \leq N$;
...
$I = N \leq N$;
$I = N + 1 \leq N$ (corte) y sale.

## Ejemplo 7: Algoritmo 1: Búsqueda del elemento máximo en una sucesión finita

```
Entrada: a[]: enteros
Salida: Máximo (a[1], a[2],...,a[N])
MAX:=a[1]
FOR I:=2 to N        Comparaciones del FOR I: N
BEGIN
 IF (MAX<a[I])        Se ejecuta N-1 veces, por lo que
 THEN MAX:=a[I]       hace N-1 comparaciones
  END
END
```

Sumando las comparaciones, obtenemos la complejidad del algoritmo: $f(N) = N + N - 1 = 2N - 1$ y podemos decir que $f \in O(N)$

## Ejemplo 8: Algoritmo 2

```
X:=0
FOR I:=1 TO N      → Comparaciones del FOR  I=N+1
BEGIN
  FOR J:=1 TO N    → Comparaciones propias del FOR J=N+1
  BEGIN
  X:=X+I           → No hay comparaciones
  END              → Este bloque se repite N veces
END
```

Sumando las comparaciones, obtenemos la complejidad del algoritmo:
$f(N) = N + 1 + N(N + 1) = N^2 + 2N + 1$ y podemos decir que $f \in O(N^2)$

## Ejemplo 9: Algoritmo 3

```
Entrada: N
Salida: N!
FACT:=1
  FOR I:=1 TO N  ⟶  Comparaciones del FOR  I=N+1
   BEGIN
   FACT:=FACT*I
   END
```

Sumando las comparaciones, obtenemos la complejidad del algoritmo: $f(N) = N + 1$ y podemos decir que $f \in O(N)$

## Ejemplo 10: Algoritmo 4 (algoritmo recursivo)

```
Entrada: un número natural
FACT(N: natural)
{
    IF (N=1)  ⟹  Esta pregunta la realiza N veces
    THEN RETURN 1
    ELSE RETURN N*FACT(N-1)
}
```

Sumando las comparaciones, obtenemos la complejidad del algoritmo: $f(N) = N$ y por lo tanto $f \in O(N)$

**Obs:** *Un algoritmo se llama* ***recursivo*** *si resuelve un problema reduciéndolo a un caso del mismo problema con datos de entrada más pequeños.*

## Ejemplo 11: Algoritmo de ordenación (Método de Selección)

```
Entrada: a[ ]: enteros
FOR I:=1 TO N                    → N+1 COMPARACIONES
BEGIN
   FOR J:=I+1 TO N               → N-1+1 COMPARACIONES
   BEGIN
    IF (a[J]<a[I])               → 1 COMPARACIÓN
    THEN
      BEGIN
      MIN:=J
      SWAP(a, I, MIN)            → SIN COMPARACIONES
      END
   END
END
```

El algoritmo busca el mínimo elemento entre una posición I y el final de la lista, luego intercambia el mínimo con el elemento de la posición I.

Antes de analizar la complejidad, veamos un ejemplo: (con N=4)

| I | J | MIN | a |
|---|---|---|---|
| 1 | 2 | | 10 15 5 2 |
| | 3 | 3 | 5 15 10 2 |
| | 4 | 4 | 2 15 10 5 |
| | 5 | | |
| **I** | **J** | **MIN** | **a** |
| 2 | 3 | 3 | 2 10 15 5 |
| | 4 | 4 | 2 5 15 10 |
| | 5 | | |
| **I** | **J** | **MIN** | **a** |
| 3 | 4 | 4 | 2 5 10 15 |
| | 5 | | |
| **I** | **J** | **MIN** | **a** |
| 4 | 5 | | |

Análisis de la complejidad: *No hay mejor ni peor caso*: <u>siempre compara</u>.

| | | # veces que entra al For J + J≤N + IF | J≤N Corte For J | I ≤ N |
|---|---|---|---|---|
| I=1 | J=2 to N | (N-2+1).2 | 1 | 1 |
| I=2 | J=3 to N | (N-3+1).2 | 1 | 1 |
| .. | .. | .. | .. | .. |
| I=N-1 | J=N to N | (N-N+1).2 | 1 | 1 |
| I=N | J=N+1 to N | - | 1 | 1 |
| I=N+1 | - | - | - | 1 |

Sumando las comparaciones, tenemos

$$f(N) = \sum_{I=2}^{N}(N - I + 1)2 + \sum_{I=1}^{N} 1 + \sum_{I=1}^{N+1} 1$$

$$f(N) = \sum_{I=2}^{N} 2N - \sum_{I=2}^{N} 2I + \sum_{I=2}^{N-1} 2 + \sum_{I=1}^{N} 1 + \sum_{I=1}^{N+1} 1$$

$$f(N) = 2N(N-1) - 2(\frac{(N-1)N}{2} + N - 1) + 2(N-1) + N + N + 1$$

$$f(N) = N^2 + N + 1$$

Así $f \in O(N^2)$.

**Recordar:**

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^{n} A = nA$$

## Ejemplo 12: Algoritmo de ordenación (Método de inserción)

```
Entrada: a[ ]: enteros
FOR I:=2 TO N          ⟶  N comparaciones
BEGIN
   V:=a[I]
   J:=I
   WHILE (a[J-1]>V AND J>1)  ⟶  2 comparaciones
   BEGIN
    a[J]:=a[J-1]
    J:=J-1
    a[J]:=V
   END
END
```

El algoritmo consiste en el recorrido por la lista seleccionando en cada iteración un valor como clave y compararlo con el resto insertándolo en el lugar correspondiente.

Veamos un ejemplo: ($N = 4$)

| I | V | J | a |
|---|---|---|---|
| 2 | 15 | 2 | 10 15 5 2 |
| I | V | J | a |
| 3 | 5 | 3 | 10 15 5 2 |
| | | 2 | 10 5 15 2 |
| | | 1 | 5 10 15 2 |
| I | V | J | a |
| 4 | 2 | 4 | 5 10 2 15 |
| | 5 | 3 | 5 2 10 15 |
| | | 2 | 2 5 10 15 |
| | | 1 | |
| I | V | J | a |
| 5 | | | |

Análisis de la complejidad: *el peor caso es cuando la cadena de entrada está ordenada de mayor a menor*.

| | | #Comp. While | #Comp. For |
|---|---|---|---|
| I=2 | J=2 | 2.2 | 1 |
| I=3 | J=3 | 2.3 | 1 |
| ... | ... | ... | |
| I=N | J=N | 2N | 1 |
| I=N+1 | | | 1 |

En este caso:

$$f(N) = \sum_{I=2}^{N} 2I + \sum_{I=2}^{N+1} 1 = 2(\frac{(N+1)N}{2} - 1) + N$$

$$f(N) = N^2 + 2N - 2$$

Entonces: $f \in O(N^2)$.

## **Ejercicio**: Calcular la complejidad e indicar el orden

```
X:=0
FOR I:=1 TO N
BEGIN
   FOR J:=1 TO N
   BEGIN
      FOR K:=1 TO N
      BEGIN
      X:=X+1
      END
   END
END
```